Neural ODE Mini-Batching

A Step-by-Step Guide to Problems and Solutions

From shared trajectories to irregular, sample-specific time grids

Goal. Train a neural vector field using short trajectory windows while preserving the correct dynamics and observation timestamps. We progressively relax three assumptions: (i) full trajectories are used, (ii) the vector field is autonomous, and (iii) all samples share one time grid. Code snippets are excerpts (occasionally shortened for readability) from the two accompanying notebooks. The final grouping extension is *illustrated*, not fully implemented.

Source notebooks: `minibatches_torchdiffeq.ipynb` (basic batching and windows) and `irregular_time_minibatching.ipynb` (non-autonomous dynamics and irregular grids). Cell indices below are **zero-based**.

1 Problem 1: How do we train on multiple trajectories at once?

Problem — One ODE solve per trajectory is inefficient

We observe N trajectories $z^{(i)}(t)$ generated from different initial conditions but governed by a shared autonomous dynamics model,

$$\dot{z}(t) = f_\theta(z(t)), \qquad z^{(i)}(t_0) = z_0^{(i)}.$$

If every trajectory has the same evaluation timestamps $t_0, \ldots, t_{T-1}$, solving them one by one wastes the opportunity for tensor batching.

Solution — Batch across the initial-condition dimension

Stack B starting states into $Z_0 \in \mathbb{R}^{B\times d}$ and solve once:

$$\widehat{Z} = \mathrm{odeint}(f_\theta, Z_0, \mathbf{t}) \in \mathbb{R}^{T\times B\times d}.$$

The ODE solver calls the same network on the whole state batch; it does not mix the independent trajectories.

Code snapshot

```
initial_states = batch[:, 0, :]  # [B, d]
predictions = solve(model, initial_states,
                    t_window_device, **SOLVER_KWARGS)
predictions = predictions.permute(1, 0, 2)  # [B, L, d]
loss = nn.functional.mse_loss(predictions, batch)
```

Notebook reference: `minibatches_torchdiffeq.ipynb`, code cell 10, function `train_step`; cell 8 defines `NeuralVectorField`.

Key limitation. A standard call to `odeint` accepts only one evaluation-time vector for the entire batched state.

2 Problem 2: Can we train using short pieces rather than entire trajectories?

Problem — Full-horizon integration can be expensive

Instead of using every observation in a trajectory, use a window beginning at index j with L successive observations,

$$W_{i,j} = (z^{(i)}(t_j), \ldots, z^{(i)}(t_{j+L-1})).$$

One window is one training example: its first observed state is the initial condition, and subsequent states are targets.

Solution — Window sampling and relative-time integration

For an *autonomous* vector field, shifting the time origin does not change the dynamics. If windows lie on a common regular grid, they share the relative grid

$$\boldsymbol{\tau} = (0, \Delta t, \ldots, (L-1)\Delta t), \qquad \dot{z} = f_\theta(z).$$

Thus windows starting at times 2 and 10 can both be integrated over $[0, 1, 2, 3]$ (for $\Delta t = 1$), using different initial states.

Code snapshot

```
class TrajectoryWindowDataset(Dataset):
    def __getitem__(self, idx):
        traj_idx, start = divmod(idx, self.starts_per_trajectory)
        return self.paths[traj_idx, start:start + self.window_len]

train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE,
                          shuffle=True, drop_last=False)
t_window = t_grid[:WINDOW_LEN] - t_grid[0]
```

Notebook reference: `minibatches_torchdiffeq.ipynb`, code cell 6, `TrajectoryWindowDataset`; cell 10, `train_step`.

New limitation. Shifting all windows to zero only preserves the original dynamics automatically when f_θ does *not* depend explicitly on physical time. The windows must also have the same relative evaluation grid to be directly batched.

3 Problem 3: What if the dynamics explicitly depend on time?

Problem — Naively resetting clocks changes a non-autonomous model

Now consider

$$\dot{z} = f_\theta(z, t).$$

A window starting at physical time 2 and another starting at 10 can follow different dynamics even if their initial states agree. A relative solver clock $\tau = 0$ cannot be passed into the network as if it were the absolute time for both samples.

Solution — First case: same physical start and same irregular clock

Irregular spacing alone is not an obstacle. If a batch uses the *same* slice of one shared irregular time grid, pass that actual (absolute) grid directly to the solver.

Code snapshot

```
def shared_window_step(model, data, times, start, trajectory_indices):
    y = data[trajectory_indices, start:start+L]
    absolute_t = times[start:start+L]
    pred = odeint(model, y[:,0], absolute_t, method="rk4",
                  options={"step_size": 0.075}).permute(1,0,2)
    return (pred - y).square().mean()
```

Notebook reference: `irregular_time_minibatching.ipynb`, code cell 7, `shared_window_step`.

Solution — General case: shared relative grid, different physical start times

For sample b, let s_b be its starting *physical* time and let τ denote elapsed time within its window. Integrate

$$\boxed{\frac{dz_b}{d\tau} = f_\theta(z_b, s_b + \tau)}.$$

The solver sees one relative grid, while the network receives a different absolute time for every batch member.

Code snapshot

```
class OffsetTimeField(nn.Module):
    def __init__(self, field, starts):
        super().__init__()
        self.field = field
        self.starts = starts    # shape [B]

    def forward(self, tau, z):
        absolute_t = self.starts + tau
        return self.field(absolute_t, z)
```

Notebook reference: `irregular_time_minibatching.ipynb`, code cell 11, `OffsetTimeField`; cell 7, `TimeDependentField` accepts per-sample times.

Example. Starts $s_A = 2$ and $s_B = 10$; at relative time $\tau = 1$, the field receives physical times 3 and 11. **Remaining limitation:** the batched windows still need the same relative observation grid.

4 Problem 4: What if every window has its own irregular grid?

Consider three windows with different physical starts and relative timestamps:

$$\boldsymbol{\tau}_A = (0, 0.5, 1, 2),$$
$$\boldsymbol{\tau}_B = (0, 0.3, 1, 2),$$
$$\boldsymbol{\tau}_C = (0, 0.5, 1.5, 2).$$

A single standard `odeint` call cannot receive three different time vectors. The next strategies differ in *how they organize numerical computation*, not in the underlying learning objective.

4.1 Strategy A — Separate integrations, one optimizer update

Intuition. Give each window its own clock and solver call. Combine the resulting losses and update the network once for the full mini-batch.

For a mini-batch of B windows, sample b with L_b observations has prediction $\hat{z}_b(t_{b,j})$. The notebook computes an equal weight per window:

$$\mathcal{L}_A(\theta) = \frac{1}{B}\sum_{b=1}^{B}\left[\frac{1}{L_b d}\sum_{j=1}^{L_b}\|\hat{z}_b(t_{b,j}) - z_b(t_{b,j})\|_2^2\right].$$

This is *mini-batched optimization*, even though solver integration is not vectorized.

Code snapshot

```
def loss_separate(field, windows, device=DEVICE):
    losses = []
    for t, y in windows:
        t, y = t.to(device), y.to(device)
        pred = odeint(field, y[0], t, method='rk4',
                      options={'step_size': 0.08})
        losses.append((pred-y).square().mean())
    return torch.stack(losses).mean()

loss = loss_separate(model_A, batch)
opt_A.zero_grad(); loss.backward(); opt_A.step()
```

Notebook reference: `irregular_time_minibatching.ipynb`, code cell 16, `loss_separate`.

Benefit: simple, robust, supports arbitrary observation times. **Cost:** B calls to `odeint` per training batch, with limited cross-sample vectorization.

4.2 Strategy B — Union-grid vectorization

Intuition. Create one larger relative grid containing every observation time used by any window, integrate all samples on that grid, then keep only the predictions where each sample has real observations.

Define $s_b = t_{b,0}$ and $\tau_{b,j} = t_{b,j} - s_b$. Construct

$$U = \operatorname{sort}\left(\bigcup_{b,j}\{\tau_{b,j}\}\right).$$

For the three-window example,

$$U = (0, 0.3, 0.5, 1, 1.5, 2).$$

Integrate $\dot{z}_b(\tau) = f_\theta(z_b(\tau), s_b + \tau)$ simultaneously at the points in U. Define $k(b,j)$ by $U_{k(b,j)} = \tau_{b,j}$. The loss is evaluated *only* at the observed timestamps:

$$\mathcal{L}_B(\theta) = \frac{1}{BLd}\sum_{b=1}^{B}\sum_{j=1}^{L}\left\|\hat{z}_b(U_{k(b,j)}) - z_b(t_{b,j})\right\|_2^2.$$

The displayed formula matches the notebook's equal window length L; variable-length windows would need an appropriate mask or per-window averaging.

Code snapshot

```
starts = torch.stack([t[0] for t in t_list])
rel = torch.stack([t - t[0] for t in t_list])   # [B, L]
targets = torch.stack(y_list)                    # [B, L, d]
initial = targets[:, 0, :]                      # [B, d]

union, inverse = torch.unique(rel.reshape(-1), sorted=True,
                              return_inverse=True)
sample_grid_indices = inverse.reshape(rel.shape)
all_predictions = odeint(OffsetTimeField(field, starts),
                         initial, union, method='rk4',
                         options={'step_size': 0.08})  # [U, B, d]
pred = all_predictions[sample_grid_indices,
                       torch.arange(len(windows), device=device)[:, None]]
return (pred-targets).square().mean()
```

Notebook reference: `irregular_time_minibatching.ipynb`, code cell 19, `loss_union_grid`; compare with `loss_separate` in the same cell.

Benefit: one batched solver call, no need to interpolate or move observations onto approximate times. **Cost:** potentially many requested output timestamps, intermediate outputs for samples without observations, and possible coupling of adaptive solver step selection across the batch. The notebook uses fixed-step RK4.

4.3 Strategy C — Group compatible windows

Intuition. Do not force all windows into one huge batch. First collect windows with identical (or similar) relative clocks, then perform one batched integration per group.

Exact compatibility. If $\boldsymbol{\tau}_A = \boldsymbol{\tau}_B$, they can share a standard relative-time solve while retaining separate absolute offsets. For example,

$$\begin{aligned}\mathbf{t}_A &= (2, 2.3, 2.9, 3.4),\\ \mathbf{t}_B &= (5.1, 5.4, 6.0, 6.5),\\ \mathbf{t}_C &= (5.1, 5.6, 6.0, 6.5).\end{aligned}$$

A and B share the same relative grid $(0, 0.3, 0.9, 1.4)$, unlike C.

Code snapshot

```
def same_relative_grid(a, b):
    return (len(a) == len(b) and
            torch.allclose(a-a[0], b-b[0],
                           atol=1e-6, rtol=0))
```

Notebook reference: `irregular_time_minibatching.ipynb`, code cell 22, `same_relative_grid` (compatibility demonstration only).

Approximately compatible grids. For two nearby relative grids, e.g. $(0, 0.3, 0.9, 1.4)$ and $(0, 0.35, 0.95, 1.4)$, one can *group* them but still use the *exact union grid within the group*. Do not replace measured timestamps with rounded ones merely to simplify batching.

$$\text{Windows} \longrightarrow \text{Groups} \longrightarrow \text{One (union-)grid solve per group} \longrightarrow \text{Aggregate loss.}$$

Status. The notebook provides a test for identical relative grids but *does not implement a full grouping algorithm or groupwise training loop.* The group-by-similarity + union-grid variant is a natural extension. It may reduce the cost of heterogeneous grids, but this is workload- and solver-dependent.

5 How do the strategies compare?

	A: Separate	**B: Union grid**	**C: Grouped**
Solver calls	One per window	One per batch	One per group
Vectorization	No across windows	Yes, entire batch	Yes, within groups
Different clocks	Fully supported	Fully supported	Supported groupwise
Primary overhead	Many solves	Large union / unused outputs	Grouping + multiple solves
Implementation	Full, cell 16	Full, cell 19	Compatibility demo, cell 22

Important numerical subtlety. The number of *requested output timestamps* need not equal the number of *internal integration steps.* With adaptive solvers (such as `dopri5`), the internal step count depends on error control and the dynamics; extra output times do not necessarily imply proportionally more network evaluations. With fixed-step RK4, cost instead depends strongly on step size and integration duration. Benchmark actual wall time, memory, numerical error, and gradients rather than assuming one method is universally fastest.

6 What is invariant across all three strategies?

Each strategy minimizes a data-fit objective over short windows for the *same* time-dependent vector field f_θ. With equal-length windows and matching numerical accuracy, their loss definitions agree in the exact-solution limit. What changes is the arrangement of solver calls and tensors, potentially introducing different *numerical errors* or runtime costs.

A final warning: a small loss on short windows does not guarantee good predictions over long horizons, because local errors can accumulate during full-trajectory rollouts. The irregular-time notebook explicitly evaluates this distinction after its initial window-training experiment (code cell 9).

Reading map. Basic batch and windows: `minibatches_torchdiffeq.ipynb`, cells 6, 8, 10. Non-autonomous shared windows and offsets: `irregular_time_minibatching.ipynb`, cells 7, 11. Distinct grids: cells 13–14. Strategies A, B, C: cells 16, 19, 22. Illustrative benchmark: cell 24.